

Q. Explain Comments, Keywords, Data Types, Variables, Operators, Control Statement and Iterative statements in javascript
Ans.

Comments

Comments are notes in the code that JavaScript ignores. They help you or others understand the code.

- **Single-line comment:** `// This is a comment`
- **Multi-line comment:**

Keywords:

Keywords are reserved words that have special meaning in JavaScript. You can't use them as variable names.

Examples: `let`, `const`, `if`, `else`, `function`, `return`, etc.

Data Types

Data types define the type of value a variable holds. Main types:

- **String:** Text (e.g., "hello")
- **Number:** Numbers (e.g., 10, 3.14)
- **Boolean:** true or false
- **Undefined:** A variable declared but not assigned a value
- **Null:** No value
- **Object:** Complex data (e.g., {name: "John"})
- **Array:** List of values (e.g., [1, 2, 3])

Variables

Variables store data. You can declare them using:

- `var` (old way)
- `let` (modern and preferred)
- `const` (for values that don't change)

Operators

Operators perform actions on variables and values:

- **Arithmetic:** `+`, `-`, `*`, `/`, `%`
- **Assignment:** `=`, `+=`, `-=`
- **Comparison:** `==`, `===`, `!=`, `>`, `<`
- **Logical:** `&&` (and), `||` (or), `!` (not)

Control statements:

These control the flow of your code.

if / else: Runs code based on conditions

switch: Checks multiple values

Iterative Statements (Loops)

These repeat actions.

For loop

While loop

Do-while loop

Q. Explain explain Java Script Architecture
Ans.

JavaScript runs in a single thread, meaning it executes one task at a time, but it can handle asynchronous operations using the browser's features.

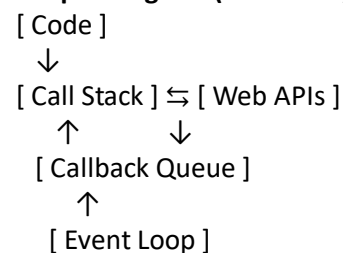
Key Parts:

1. **Call Stack** – Where functions run one by one.
2. **Heap** – Where memory (like objects) is stored.
3. **Web APIs** – Browser-provided features (like `setTimeout`, `DOM`, `fetch`) that handle async tasks.
4. **Callback Queue** – Holds tasks waiting to run after Web APIs finish.
5. **Event Loop** – Continuously checks the Call Stack. If it's empty, it moves tasks from the Callback Queue into the Call Stack.

How It All Works Together

1. You write some JavaScript code.
2. Functions run and go on the **Call Stack**.
3. If an async task is called (like `setTimeout`), it's handled by **Web APIs**.
4. Once finished, the callback goes to the **Callback Queue**.
5. The **Event Loop** checks the stack and moves the callback when ready.

Simple Diagram (Text Form)



Q. Explain Framework and Libraries in javascript

Ans.

Library

- A **collection of pre-written code** that helps you do specific tasks.
- You **call** functions from a library when you need them.
- **You control the flow** of your app.

Example:

- **jQuery** – Makes DOM manipulation easier.
- **Lodash** – Helps with array and object operations.

Framework

- A **complete structure** for building applications.
- It **controls the flow** and calls **your code** at certain points.
- Frameworks usually include tools for UI, routing, state management, etc.

Example:

- **React** (library, often treated like a framework)
- **Angular, Vue.js, Next.js** (frameworks)

Main Difference:

- **Library:** You're in charge.
- **Framework:** It's in charge, and you plug your code into it.

Q. Explain Java Script Objects – Object, Date, String, Array, Math, Number, Boolean

Ans.

Object

- A collection of **key-value pairs**.
- Used to store related data.

```
let person = {  
  name: "Alice",  
  age: 25  
};
```

Date

- Used to work with **dates and times**.

```
let now = new Date();
```

String

- Represents **text**.
- Has helpful methods like `.length`, `.toUpperCase()`, `.slice()`.

```
let message = "Hello";  
console.log(message.toUpperCase());
```

Array

- Stores **multiple values** in a single variable.

```
let fruits = ["apple", "banana", "mango"];  
console.log(fruits[1]);
```

Math

- Provides **math functions** like rounding, random numbers, etc.

```
console.log(Math.random()); // Random  
number between 0 and 1  
console.log(Math.round(4.7));
```

Number

- Handles numeric values and has methods like `.toFixed()`.

```
let num = 10.456;
```

```
console.log(num.toFixed(2));
```

Boolean

- Represents **true** or **false** values.

```
let isOnline = true;
```

These built-in objects help you handle different types of data easily in JavaScript.

Q. explain event handling-Mouse, Keyboard, Form, Window in javascript

Ans.

Mouse Events

These happen when the user uses the mouse.

- **Examples:**
 - **click** – when an element is clicked
 - **dblclick** – double click
 - **mouseover** – when mouse moves over an element
 - **mouseout** – when mouse leaves an element

```
element.addEventListener("click", function()  
{  
  alert("Clicked!");  
});
```

Keyboard Events

These happen when a user presses a key.

- **Examples:**
 - **keydown** – when a key is pressed down
 - **keyup** – when a key is released
 - **keypress** – (mostly outdated)

Q. explain BOM –Object Form Validations, Regular Expressions

Ans.

BOM (Browser Object Model)

- BOM lets JavaScript **interact with the browser** (not the webpage itself).
- Main BOM objects:
 - window – Main browser window
 - navigator – Info about the browser
 - screen – Info about the screen
 - location – URL-related info
 - history – Browsing history

```
console.log(window.location.href);
```

Form Validations

- Checks if user input is correct **before submitting** a form.

Examples:

- Required fields
- Email format
- Password length

```
function validateForm() {  
  let name =  
  document.forms["myForm"]["username"].value;  
  if (name === "") {  
    alert("Name must be filled out");  
    return false;  
  }  
}
```

Regular Expressions (RegEx)

- Patterns used to **match or search text** (very useful in validations).

Example:

- Validate email format:

```
let email = "test@example.com";  
let pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
console.log(pattern.test(email));
```

In Short:

- **BOM** = controls browser (window, location, etc.)
- **Form Validation** = checks user input
- **RegEx** = powerful tool for matching patterns in strings

Q. explain Asynchronous JavaScript-Callbacks, Promises, and Async/Await Managing Asynchronous Data Flow Working with Timers and Intervals Handling HTTP Requests with Fetch API

Ans:

Asynchronous JavaScript

JavaScript runs one line at a time, but **asynchronous code** lets it handle tasks like API calls, delays, etc., **without blocking** the rest of the code.

Callbacks

A function passed into another function to run **after a task is done**.

```
setTimeout(function() {  
  console.log("This runs later");  
}, 1000);
```

Promises

Promises are used to handle async tasks. They have:

- resolve (success)
- reject (failure)

```
fetch('url')  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.log(error));
```

Async/Await

A cleaner way to work with promises.

```
async function getData() {  
  let response = await fetch('url');  
  let data = await response.json();  
  console.log(data);  
}
```

Timers and Intervals

- `setTimeout()` – runs code **once after a delay**
- `setInterval()` – runs code **repeatedly at intervals**

```
setTimeout(() => console.log("After 1 sec"),  
1000);  
setInterval(() => console.log("Every 1 sec"),  
1000);
```

Fetch API

Used to make **HTTP requests** (like getting data from a server).

```
fetch("https://api.example.com/data")  
  .then(res => res.json())  
  .then(data => console.log(data));
```

Q. explain Event Handlers and Callback Functions in javascript

Ans.

Event Handlers

- Event handlers are **functions that run when something happens**, like a click, key press, or form submit.

```
document.getElementById("btn").addEventListener("click", function() {
    alert("Button clicked!");
});
```

Callback Functions

- A **callback** is a function passed **into another function** to run **later**, often after a task finishes.

```
function greet(name, callback) {
    console.log("Hi " + name);
    callback();
}
```

```
greet("Alice", function() {
    console.log("This is a callback!");
});
```

The second function runs **after** the greeting — that's a **callback**.

☐ **Event Handler** = Responds to user actions (like clicks).

☐ **Callback Function** = Passed to another function to run **later** (often after something is done).

Q. explain WebSockets in javascript

Ans.

WebSockets let the browser and server **communicate instantly in both directions**, without reloading the page.

Unlike normal HTTP requests (which are one-way), **WebSockets keep a connection open**, so data can flow back and forth **in real time**.

Use Cases

- Chat apps
- Live notifications
- Online games
- Real-time dashboards
- **In Simple Words:**
- WebSockets allow **live communication** between the browser and server — perfect for real-time apps.

```
let socket = new
WebSocket("wss://example.com/socket");
```

```
socket.onopen = function() {
    console.log("Connection opened");
    socket.send("Hello Server!");
};
```

```
socket.onmessage = function(event) {
    console.log("Message from server:",
event.data);
};
```

```
socket.onclose = function() {
    console.log("Connection closed");
};
```

Q. explain Creating and Using JavaScript Classes, Object Prototypes, Inheritance Patterns, Encapsulation Techniques, Polymorphism and Code Reusability

Ans.

- A **class** is like a **blueprint** for creating objects with shared properties and methods.

```
class Car {
    constructor(brand, model) {
        this.brand = brand;
        this.model = model;
    }

    displayInfo() {
        console.log(this.brand + " " + this.model);
    }
}
```

```
let myCar = new Car("Toyota", "Camry");
myCar.displayInfo();
```

Object Prototypes

- Every object in JavaScript has a **prototype**, which is a template for methods and properties that the object can access.

```
let car = { brand: "Toyota" };
console.log(car.toString());
```

Inheritance Patterns

- **Inheritance** allows one class (child) to inherit properties and methods from another class (parent).

```
class Animal {  
  speak() {  
    console.log("Animal speaks");  
  }  
}
```

```
class Dog extends Animal {  
  speak() {  
    console.log("Dog barks");  
  }  
}
```

```
let dog = new Dog();  
dog.speak();
```

Polymorphism

- **Polymorphism** allows objects of different types to be treated as if they were the same type, usually by using the same method name.

```
class Cat {  
  sound() {  
    console.log("Meow");  
  }  
}
```

```
class Dog {  
  sound() {  
    console.log("Bark");  
  }  
}
```

```
let animals = [new Cat(), new Dog()];  
animals.forEach(animal => animal.sound());
```

Code Reusability

- **Code Reusability** is achieved by writing **generic classes and methods** that can be used in multiple places, reducing duplication.

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
  
  greet() {  
    console.log("Hello, " + this.name);  
  }  
}
```

```
}  
}
```

```
let person1 = new Person("Alice");  
let person2 = new Person("Bob");
```

```
person1.greet(); // Hello, Alice  
person2.greet();
```

In Short:

- **Classes:** Templates for creating objects.
- **Prototypes:** Share methods and properties between objects.
- **Inheritance:** Child objects can inherit from parent objects.
- **Encapsulation:** Hides internal details and exposes only what's necessary.
- **Polymorphism:** Same method names, different behaviors.
- **Code Reusability:** Reuse code through shared classes/methods.

Q.explain JSON - Iterators and generators

Working with Iterators, Working with Generators in javascript

Ans.

JSON (JavaScript Object Notation)

- **JSON is a format for storing and exchanging data (often used to send data between a server and a client).**
- **It's text-based and looks like a JavaScript object, but it must be in a string format.**

Example:

```
let person = { name: "Alice", age: 25 };
```

```
// Convert JavaScript object to JSON string  
let jsonString = JSON.stringify(person);
```

```
// Convert JSON string back to JavaScript object  
let parsedPerson = JSON.parse(jsonString);
```

Iterators

- Iterators are **objects** that allow you to loop through a collection (like an array or object).
- Every iterable object (e.g., arrays) has an iterator with a `next()` method to get the next item.

Example:

```
let numbers = [1, 2, 3];
let iterator = numbers[Symbol.iterator]();
```

```
console.log(iterator.next().value); // 1
console.log(iterator.next().value); // 2
console.log(iterator.next().value); // 3
```

Generators

- A **generator** is a special function that can **pause** and **resume** execution.
- It uses function* and yield to give values one by one.

Example:

```
function* count() {
  yield 1;
  yield 2;
  yield 3;
}
```

```
let gen = count();
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
console.log(gen.next().value); // 3
```

In Short:

- **JSON:** Used for **storing and exchanging data** (object to string and vice versa).
- **Iterators:** Let you **loop** through collections one item at a time.
- **Generators:** Functions that can **pause and resume**, yielding values on demand.

Q. explain Document Object Model, Document structure, selecting document elements and query selectors, moving thorough DOM tree in javascript

Ans.**Document Object Model (DOM)**

- The DOM represents the HTML document as a tree of objects where each object is a part of the page (like elements, text, etc.).
- JavaScript can interact with the DOM to modify HTML elements, style them, or respond to user events

Document Structure

- The structure of a web page in the DOM looks like a **tree** where each node represents an HTML element.

Example (HTML):

```
html
<!DOCTYPE html>
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Hello World</h1>
    <p>Welcome to the page.</p>
  </body>
</html>
```

Selecting Document Elements

JavaScript provides methods to select elements in the DOM.

Examples:

- getElementById() – Select by ID
- getElementsByClassName() – Select by class
- querySelector() – Select the first matching element (CSS selectors)

```
let heading =
document.getElementById("myHeading");
let paragraphs =
document.getElementsByClassName("myClass");
let firstPara = document.querySelector("p");
```

Q. explain HTML elements and attributes, Creating, changing and deleting nodes in javascript

Ans.

HTML Elements and Attributes

- **HTML Elements** are the building blocks of an HTML document (like <div>, <p>, <h1>, etc.).
- **Attributes** are extra information inside an element (like id, class, src, etc.).

Example:

```
<div id="myDiv" class="box">Hello  
World</div>
```

Creating, Changing, and Deleting Nodes in JavaScript

1. Creating Nodes

You can create new HTML elements using createElement() and add content with createTextNode().

```
let newDiv = document.createElement("div");  
// Create a new div  
let text = document.createTextNode("Hello,  
World!"); // Create text node  
newDiv.appendChild(text); // Add text to the  
div  
document.body.appendChild(newDiv); // Add  
the new div to the page
```

2. Changing Nodes

You can change the content or attributes of an element using properties like .innerHTML or .setAttribute().

```
let div = document.getElementById("myDiv");  
div.innerHTML = "New Content!"; // Change  
the content inside the div  
div.setAttribute("class", "newClass"); //  
Change the class attribute
```

3. Deleting Nodes

To delete an element, use .removeChild() or .remove().

```
let divToDelete =  
document.getElementById("myDiv");  
divToDelete.remove(); // Remove the div from  
the DOM
```

In Short:

- **HTML Elements** are the tags in the page, and **Attributes** are extra info about those tags.

- You can **create** elements with createElement(), **change** content/attributes with .innerHTML or .setAttribute(), and **delete** elements with .remove().

Q.explain JSON in javascript

Ans.

JSON stands for JavaScript Object Notation. It is a lightweight format used to store and share data, especially between a server and a web page.

JSON Example:

```
{  
  "name": "Alice",  
  "age": 25,  
  "isStudent": true  
}
```

This looks like a JavaScript object but it's just a string when used in JSON.

Converting Between JSON and JavaScript

-  **JavaScript to JSON** (object → string):

```
let person = { name: "Alice", age: 25 };  
let jsonString = JSON.stringify(person);  
console.log(jsonString);
```

- **JSON to JavaScript** (string → object):

```
let jsonData = '{"name": "Alice", "age": 25}';  
let person = JSON.parse(jsonData);  
console.log(person.name);
```

In Short:

- **JSON is a text format used to store or send data.**
- **Use JSON.stringify() to convert JavaScript data to JSON.**
- **Use JSON.parse() to convert JSON text to JavaScript.**

Q. explain node

Ans.

Node.js is a **JavaScript runtime** that lets you run JavaScript **outside the browser**, usually on a **server**.

- Built on **Chrome's V8 engine** (the same engine that runs JavaScript in Chrome).
- Allows you to build **server-side** applications (like APIs, file handling, databases, etc.).

What Can Node.js Do?

- Read/write files
- Build web servers and APIs
- Connect to databases
- Handle real-time data (like chat apps)

In short:

Node.js = JavaScript for the server

It lets you build powerful backend applications using JavaScript.

```
const http = require('http');
```

```
const server = http.createServer((req, res) => {  
  res.write('Hello from Node.js!');  
  res.end();  
});
```

```
server.listen(3000, () => {  
  console.log('Server running on  
http://localhost:3000');  
});
```

This creates a simple server that shows “Hello from Node.js!” in the browser at **localhost:3000**.

Q. explain React environment, Create React App

Ans.

React Environment

- The React environment is the setup needed to build and run React apps.
- It includes:
 - Node.js (to run JavaScript outside the browser)
 - npm or yarn (to install packages)
 - A bundler like Webpack (to combine code files)
 - Babel (to convert modern JS/JSX into browser-compatible code)

Create React App (CRA)

- A tool that sets up a React project automatically.
- It gives you a ready-to-use structure with all tools pre-configured.

To create a React app:

```
npx create-react-app my-app
```

```
cd my-app
```

```
npm start
```

✓ Your app runs on `http://localhost:3000`

In Short:

- **React environment** = tools needed to run a React app.
- **Create React App** = easiest way to start a new React project without setup headaches.

React Folder Structure:

```
my-app/
```

```
├── public/    → Static files (like index.html, images)
```

```
├── src/       → Your React code (JSX, CSS, components)
```

```
|   ├── App.js → Main component
```

```
|   ├── index.js → Entry point, renders App
```

```
└── package.json → Project info & dependencies
```

In Short:

- **public/** → Static content
- **src/** → All your React code
- **App.js** → Main UI logic
- **index.js** → Starts the app

Q. explain Hello World, Components, JSX in short and easy language

Ans.

Hello World in React

A simple first React app that displays text:

```
function App() {  
  return <h1>Hello, World!</h1>;  
}
```

Components

- Components are **building blocks** of a React app.
- They are **reusable functions** that return JSX (UI).

```
function Welcome() {  
  return <h2>Welcome to React</h2>;  
}
```

JSX (JavaScript XML)

- JSX looks like HTML but is **used in JavaScript**.
- It's how you write the UI inside React components.

Example:

```
const element = <p>This is JSX!</p>;
```

In Short:

- **Hello World** = first simple React app.
- **Component** = a function that returns UI.
- **JSX** = syntax that looks like HTML, used in React to define elements.

Q. explain Functional vs class components, Props in react

Ans.

Functional Component (Modern and preferred)

- A plain JavaScript function that returns JSX.

```
function Greet() {  
  return <h1>Hello</h1>;  
}
```

Class Component (Older style)

- Uses a class and must extend `React.Component`.

```
class Greet extends React.Component {  
  render() {  
    return <h1>Hello</h1>;  
  }  
}
```

Props (Properties)

- **Props** are used to **pass data** from one component to another.

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

// Usage:

```
<Welcome name="Alice" />
```

Output: "Hello, Alice"

In Short:

- **Functional components** are simple functions (preferred).
- **Class components** are older, use `render()`.
- **Props** = data passed to components (like function arguments).

Code Example:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

// Usage:

```
<Welcome name="Alice" />
```

Output: **Hello, Alice**

✓ Functional component

✓ Uses **props** to pass data

Q. explain State, Lifecycle methods in react

Ans.

State

- State is data that changes over time in a component.
- Used in functional components with `useState`.

```
import { useState } from 'react';
```

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return <button onClick={() => setCount(count + 1)}>Count: {count}</button>;  
}
```

- Lifecycle methods run at different stages of a component's life (like `mount`, `update`, `unmount`).

```
class MyComponent extends  
  React.Component {  
  componentDidMount() {
```

```
// runs once when component appears
}
```

```
componentWillUnmount() {
  // runs when component is removed
}
}
```

In **functional components**, we use `useEffect()` instead.

In Short:

- **State** = dynamic data in a component.
- **Lifecycle methods** = special functions that run at different stages of a component's life.

Q. explain Hooks – `useState`, `useEffect`, `useContext`

Ans:

`useState`

- Lets you create state in a functional component.

```
const [count, setCount] = useState(0);
```

→ count is the current value

→ setCount is used to update it

`useEffect`

- Runs **side effects** (like fetching data, timers) after render.

```
useEffect(() => {
  console.log("Component mounted");
}, []);
```

→ Runs once when the component loads

`useContext`

- Lets you **use shared data** (like global state) without passing props manually.

```
const value = useContext(MyContext);
```

→ Gets value from a context provider

In Short:

- `useState` → for state
- `useEffect` → for side effects
- `useContext` → for global/shared data

Q. explain Event handling in react

Ans.

Event Handling in React

- React handles **events** like click, submit, change, etc., using **camelCase** syntax.

Example:

```
function Button() {
  const handleClick = () => {
    alert('Button clicked!');
  };
}
```

```
return <button onClick={handleClick}>Click Me</button>;
}
```

Key Points:

- **Event names** are written in camelCase (e.g., `onClick`, `onChange`).
- **Event handlers** are functions that are called when the event happens.
- You **bind** the event handler to the element (e.g., `<button onClick={handleClick}>`).

In Short:

- **React events** use camelCase.
- Use functions to handle events like `onClick` or `onSubmit`.

Q. explain Forms – controlled components, submission, validation in frameworks

Ans.

Controlled Components

- In React, **controlled components** are forms where the form data is handled by **React state**.

```
function MyForm() {
  const [input, setInput] = useState("");
```

```
const handleChange = (event) => {
  setInput(event.target.value);
};
```

```
return (
  <input type="text" value={input}
    onChange={handleChange} />
);
}
```

- The input field is **controlled** because its value comes from React's state (input).

Form Validation

- You can **validate form inputs** before submission.

```
function validateInput(input) {
  return input.length >= 5; // Example
  validation rule
}
```

```
function handleSubmit(event) {
  event.preventDefault();
  if (!validateInput(input)) {
    alert("Input is too short!");
  } else {
    alert("Form submitted!");
  }
}
```

In Short:

- **Controlled components:** Form data comes from React state.
- **Form submission:** Handle with onSubmit and prevent default behavior.
- **Form validation:** Check data before allowing form submission.

Q. explain Conditional rendering – if, ternary, && in frameworks

Ans.

In React, you can show different UI elements based on certain conditions using different approaches.

Using if Statement

- You can use the if statement before the return to determine what to render.

```
function Welcome(props) {
  if (props.isLoggedIn) {
    return <h1>Welcome, User!</h1>;
  } else {
    return <h1>Please Log In</h1>;
  }
}
```

Using Ternary Operator (? :)

- The **ternary operator** is a shorthand for if-else and is often used directly inside JSX.

```
function Welcome(props) {
  return (
    <h1>{props.isLoggedIn ? "Welcome, User!" :
    "Please Log In"}</h1>
  );
}
```

Using && (Logical AND)

- The && operator is used to **render** something only if the condition is true.

```
function Welcome(props) {
  return (
    <div>
      {props.isLoggedIn && <h1>Welcome,
      User!</h1>}
    </div>
  );
}
```

In Short:

- **if:** Use for full condition checks before the return.
- **Ternary (? :):** Shorthand for if-else inside JSX.
- **&&:** Render something only if the condition is true.

Q. explain Lists and keys, Importance of keys in framework

Ans.

Lists in React

- In React, you can **render a list of items** by using the map() function to loop over an array and return a component for each item.

```
const fruits = ['Apple', 'Banana', 'Cherry'];
```

```
function FruitList() {
  return (
    <ul>
      {fruits.map((fruit, index) => <li
      key={index}>{fruit}</li>)}
    </ul>
  );
}
```

Importance of Keys

- **Keys** are special **props** used in **lists** to help React identify which items have changed, been added, or removed.
- **Keys** must be **unique** and stable (not based on array index, if possible).

```
{fruits.map((fruit) => <li  
key={fruit}>{fruit}</li>}}
```

- `key={fruit}` is used instead of `key={index}` to make sure each item has a unique identifier. This helps React optimize updates efficiently.

In Short:

- **Lists:** Render items from an array using `map()`.
- **Keys:** Help React track items in lists for better performance (each item must have a unique key).

Q. explain Styling – CSS, CSS Modules, CSS-in-JS in frameworks

Ans.

CSS (Cascading Style Sheets)

- **CSS** is the traditional way to style your React components.
- You create a `.css` file and link it to your component.

```
/* styles.css */  
.button {  
  background-color: blue;  
  color: white;  
}
```

```
import './styles.css';
```

```
function Button() {  
  return <button className="button">Click  
Me</button>;  
}
```

CSS Modules

- **CSS Modules** provide **scoped styles**, meaning the styles apply only to the component where they are imported.
- You import CSS files as a module, so class names are locally scoped.

```
/* Button.module.css */  
.button {  
  background-color: blue;  
  color: white;
```

```
}  
import styles from './Button.module.css';
```

```
function Button() {  
  return <button  
className={styles.button}>Click  
Me</button>;  
}
```

- This ensures the styles do not affect other components.

CSS-in-JS

- **CSS-in-JS** allows you to write CSS directly in JavaScript files using libraries like **styled-components** or **emotion**.

```
import styled from 'styled-components';
```

```
const Button = styled.button`  
  background-color: blue;  
  color: white;  
`;
```

```
function App() {  
  return <Button>Click Me</Button>;  
}
```

- CSS is written **inside** JavaScript, and styles are applied directly to the component.

In Short:

- **CSS:** Traditional way with global styles.
- **CSS Modules:** Scoped styles, avoid global conflicts.
- **CSS-in-JS:** Write CSS directly in JavaScript, often using libraries like styled-components.

Q. explain React Router – setup, routes, parameters

Ans.

React Router

Used to **navigate between pages** in a React app without reloading the page.

Setup

Install it first:

```
npm install react-router-dom
```

Then import in your app:

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
```

Routes

Define paths for different components:

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
  </Routes>
</BrowserRouter>
```

- / shows Home
- /about shows About

Parameters

You can pass dynamic values in the URL using **route parameters**:

```
<Route path="/user/:id" element={<User />} />
```

In the User component, get the id:

```
import { useParams } from 'react-router-dom';
```

```
function User() {
  const { id } = useParams();
  return <h2>User ID: {id}</h2>;
}
```

In Short:

- **Setup:** Install and import react-router-dom.
- **Routes:** Map URLs to components.
- **Parameters:** Use :param in path and useParams() to access it.

Q. explain Async/await, Promises, Fetch API in frameworks

Ans;

Promises

- A **Promise** represents a value that will be available **later** (like data from a server).

```
fetch('https://api.com/data')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

Async/Await

- A **cleaner way** to work with Promises using async and await.

```
async function getData() {
  try {
    const response = await
    fetch('https://api.com/data');
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

Fetch API

- **Built-in method to get data from APIs.**

```
fetch('https://api.com/data')
```

- It returns a Promise that you handle using .then() or await.

In Short:

- **Promise** = handles data that comes later.
- **Async/Await** = easier way to use Promises.
- **Fetch API** = used to get data from servers.

Q. explain Error handling, debugging, optimization in frameworks

Ans;

Error Handling

- Use try...catch to **catch and handle errors** in async code.

```
try {  
  // risky code  
} catch (error) {  
  console.error("Something went wrong",  
error);  
}
```

Debugging

- Use **console.log()** to check values.
- Use **browser DevTools** to inspect elements, network calls, and errors.
- Add **React Developer Tools** extension to debug component tree and props/state.

Optimization

- Avoid unnecessary re-renders with **React.memo** or **useMemo**.
- Use **lazy loading** for components (React.lazy + Suspense).
- Split code with **dynamic imports**.
- Keep components **small and reusable**.

In Short:

- **Error handling:** Use try/catch and Error Boundaries.
- **Debugging:** Use console.log() and DevTools.
- **Optimization:** Reduce re-renders, use memoization, lazy loading.